super keyword

1. 'super' is a keyword and it will be used to refer the members of immediate super class .
2. 'super' cannot be accessed from static context.
3. 'super' keyword can be used in three ways.
4. To access the immediate super class variable.
   Syntax: -
   super .<variable Name>
   Example: -
   super. a
   super. b
5. To access the immediate super class methods.
   Syntax: -
   super .<Method Name>
   Example: -
   super. m1 ( );
   super. m2 ( );
6. To access the immediate super class constructors.
   Syntax: -
   super .<parameter>
   Example: -

```
super( );          //Invokes immediate super class default Constructor
super (99 )        //Invokes immediate super class 1-Args Constructor
super (99,88)      //Invokes immediate super class 2-Args Constructor
```

7. When you access any variable directly then
   - ⇒ Check whether that variable is declared in the local scope or not.
   - ⇒ If found in the local scope, then local variable will be used.
   - ⇒ If not found in the local scope then check that whether that variable is declared in the class scope or not.
   - ⇒ If found, that class level variable will be used.
   - ⇒ If not found in the scope then checks whether that variable is inherited from super classes or not.
   - ⇒ If found, that inherited variable will be used.

8. When you have same name for local variables, class level variables and super class variables then do the following.
   - ⇒ Refer the local variable directly.
   - ⇒ Refer the class level variable using this keyword.
   - ⇒ Refer the super class level variable using super keyword.
9. "super" keyword can be used to access both instance and static members.
10. Assume that same variable or method available in both super class and sub class then:
    - ⇒ When you access a member with super class reference then it will access the variables and methods of super class only.
    - ⇒ When you access a member with sub class reference then it will access the variables and methods of sub class only.

11. "super" keyword can be used to access immediate super class members.
12. You cannot access indirect super class members with super keyword when same member available in immediate super class.
13. You can use methods in indirect super class to access indirect super class members.
14. You cannot access indirect super class members with super keyword when same member available in immediate super class.
15. You can use methods in indirect super class to access indirect super class members.
16. 'super' cannot be assigned to any variable.
17. When you are not writing any constructor inside the class then one default constructor will be inserted by the java compiler.
18. When you are writing any constructor inside the class the default constructor will not be inserted by the java compiler.
19. When you are not writing any statement inside the constructor then default super will be inserted by the java compiler.
20. When you are writing super statement inside the constructor then default super will not be inserted by the java compiler.
21. You can invoke super class constructor explicitly using super keyword.
22. Call to 'super' constructor must be first statement from subclass constructor.
23. Call to 'this' constructor must be first statement from subclass constructor.
24. The first statement of any constructor can be either this or super, can't be both at a time.
25. Order of constructor invocation is from subclass to super class.
26. Order of constructor execution is from super class to sub class.
27. Private members of super class cannot be accesses using super keyword also.
29. 'this' keyword must be used after invoking super class constructors only.

Program:

OOPS278.java

```
class OOPS278{
public static void main(String args[]){
Hello h=new Hello();
h.show();
   }
}
class Hai{
int a=10;
}
class Hello extends Hai{
int a=20;
void show(){
  int a=30;
  System.out.println(a);
  System.out.println(this. a);
 }
}
```

OOPS279.java

```
class OOPS279{
public static void main(String args[]){
Hello h=new Hello();
h.show();
  }
}
class Hai{
int a=10;
}
class Hello extends Hai{
int a=20;
void show(){
  int a=30;
  System.out.println(a);
  System.out.println(this.a);
  System.out.println(super.a);
  }
}
```

OOPS280.java

```
class OOPS280{
public static void main(String args[]){
Hello h=new Hello();
h.show();
  }
}
class Hai{ }
class Hello extends Hai{
int a=20;
void show(){
  int a=30;
  System.out.println(a);
  System.out.println(this.a);
  System.out.println(super.a);
 }
}
```

OOPS281.java

```
class OOPS281{
public static void main(String args[]){
Hello h=new Hello();
h.show();
 }}
class Hai{
static int a=10;
}
class Hello extends Hai{
static int a=20;
void show(){
 int a=30;
 System.out.println(a);
 System.out.println (this.  a);
  System.out.println(super. a);
 }}
```

OOPS282.java

```
class OOPS282{
public static void main(String args[]){
Hello. show();
}}
class Hai{
static int a=10;
}
class Hello extends Hai{
static int a=20;
static void show(){
int a=30;
System.out.println(a);
System.out.println(this. a);
System.out.println(super. a);
}}
```

OOPS283.java

```
class OOPS283{
public static void main(String args[]){
Hello. show();
}}
class Hai{
static int a=10;
}
class Hello extends Hai{
static int a=20;
static void show(){
int a=30;
System.out.println(a);
System.out.println(Hello.a);
System.out.println(Hai. a);
}}
```

OOPS284.java

```
class OOPS284{
public static void main(String args[]){
Hello hello=new Hello();
System.out.println(hello. a);
}
}
class Hai{
int a;
}
class Hello extends Hai{
String a;
}
```

OOPS285.java

```
class OOPS285{
public static void main(String args[]){
new C().show();
}}
class A{
int a=10;
}
class B extends A{
String a="IND";
}
class C extends B{
boolean a=true;
void show(){
float a=90.99F;
System.out.println(a);
System.out.println(this.a);
System.out.println(super.a);
}}
```

OOPS286.java

```
class OOPS286{
public static void main(String args[]){
new C().show();
}}
class A{
int a=10;
}
class B extends A{
String a="IND";
}
class C extends B{
boolean a=true;
void show(){
float a=90.99F;
System.out.println(a);
System.out.println(this. a);
System.out.println(super. a);
System.out.println(super. super. a);
}
}
```

OOPS287.java

```
class OOPS287{
public static void main(String args[]){
new C().show();
}
}
class A{
int a=10;
int getAData() {
return this. a;
}
}
class B extends A{
String a="IND";
}
class C extends B{
boolean a=true;
void show(){
float a=90.99F;
System.out.println(a);
System.out.println(this. a);
System.out.println(super. a);
System.out.println(getAData());
}
}
```

OOPS288.java

```
class OOPS288{
public static void main(String args[]){
new Hello().show();
}
}
class Hai{}
class Hello extends Hai{
void show(){
System.out.println(this);
System.out.println(super);
}
}
```

OOPS289.java

```
class OOPS289{
public static void main(String args[]){
new Hello().show();
}
}
class Hai{}
class Hello extends Hai{
void show(){
Hello h1=this;
Hai h2=super;
System.out.println(h1);
}}
```

OOPS290.java

```
class OOPS290{
public static void main(String args[]){
new Hello().show();
}}
class Hai{}
class Hello extends Hai{
void show(){
System.out.println(Hello. this);
System.out.println(Hello. super);
}
}
```

OOPS291.java

```
class OOPS291{
public static void main(String args[]){
new Hello().show();
}}
class Hai{
private int a;
}
class Hello extends Hai{
void show(){
System.out.println(a);
System.out.println(super. a);
}}
```

OOPS292.java

```
class OOPS292{
public static void main(String args[]){
new B();
 }
}
class A{
A(){
System.out.println("A->DC");
  }
}
class B extends A{
B(){
  super();
  System.out.println("B-> DC");
}}
```

OOPS293.java

```
class OOPS293{
public static void main(String args[]){
new B();
}
}
class A{
A(int a){
System.out.println("A(int) Cons");
}
}
class B extends A{
B(){
System.out.println("B-> DC");
}
}
```

OOPS294.java

```
class OOPS294{
public static void main(String args[]){
new B();
  }
}
class A{
A(int a){
System.out.println("A(int) Cons");
  }
}
class B extends A{
B(){
   super();
   System.out.println("B-> DC");
  }
}
```

OOPS295.java

```
class OOPS295{
public static void main(String args[]){
new B();
}
}
class A{
A(int a){
System.out.println("A(int) Cons");
}
}
class B extends A{
B(){
super(10);
System.out.println("B-> DC");
}
}
```

OOPS296.java

```
class OOPS296{
public static void main(String args[]){

new B();
}
}
class A{
A(int a){
System.out.println("A(int)Cons ");
}}
class B extends A{
B(){
System.out.println("B-> DC");
super(10);
}
}
```

OOPS297.java

```
class OOPS297{
public static void main(String args[]){
new B(10);
}}
class A{
A(){
System.out.println("A->DC ");
}}
class B extends A{
B()     {
System.out.println("B-> DC");
}
B(int a){
this();
System.out.println("B(int)");
}}
```

OOPS298.java

```
class OOPS298{
public static void main(String args[]){
new B(10);
}}
class A{
A(){
System.out.println("A->DC ");
}}
class B extends A{
B(){
System.out.println("B-> DC");
}
B(int a){
super();
System.out.println("B(int)");
}}
```

OOPS299.java

```
class OOPS299{
public static void main(String args[]){
new B(10);
}}
class A{
A(){
System.out.println("A->DC ");
}}
class B extends A{
B(){
System.out.println("B-> DC");
}
B(int a){
super();
this();
System.out.println("B(int)");
}}
```

OOPS300.java

```
class Java OOPS300{
public static void main(String args[]){
new B(10);
}
}
class A{
A(){
System.out.println("A->DC ");
}
}
class B extends A{
B(){
System.out.println("B-> DC");
}
B(int a){
this();
super();
System.out.println("B(int)");
}
}
```

OOPS301.java

```
class OOPS301{
public static void main(String args[]){
B bobj =new B(10,"IND");
bobj. show();
}}
class A {
int a;
A(int a){
System.out.println("A(int) Cons ");
this. a=a;
}}
class B extends A {
String a;
B(int a1,String a2){
super(a1);
this. a=a2;
System.out.println("B(int) Cons ");
}
void show(){
System.out.println(this. a);
System.out.println(super. a);
}}
```

OOPS302.java

```
class Java OOPS302{
public static void main(String args[]){
new B();
   }
}
class A  {
A(Object obj)  {
System.out.println("A(Object) Cons ");
   }
}
class B extends A {
B(){
super(this);
System.out.println("B() Cons ");
  }
}
```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS278	30 20	Here, Variable a will refer local variable and “this” will refer class level variable.
OOPS279	30 20 10	Here, Same”” but Super will refer the super class variable.
OOPS280	Compilation Error	[Error : cannot find symbol] Here, you are not declare super class variable.
OOPS281	30 20 10	‘super’ keyword can be used to access both instance and static members.
OOPS282	Compilation Error	[Error :Cannot use this in a static context Cannot use super in a static context]
OOPS283	30 20 10	We can also access variable with class name reference.
OOPS284	null	Here, Variable ‘a’ is String type in Hello so it give null.
OOPS285	90.99 true IND	Refer point 8.
OOPS286	Compilation Error	[Error: Syntax error on token "super", Identifier expected] you cannot use ‘super.super’ to access super class variable of multilevel inheritance.
OOPS287	90.99 true IND 10	Here, getAdata () method is use to access super class of multilevel inheritance.
OOPS288	Compilation Error	[Error : Syntax error on token "super", delete this token] You cannot use ‘super’ keyword to access value without any variable name.
OOPS289	Compilation Error	[Error : . expected] ‘super’ cannot be assigned to any variable.
OOPS290	Compilation Error	[Error : . expected] You cannot use ‘super’ keyword with class reference.
OOPS291	Compilation Error	[Error: ‘a’ has private access in Hai] private member of super class will not access by subclass.
OOPS292	A->DC B-> DC	Refer point 17, 18 and 19.
OOPS293	Compilation Error	[Error: Implicit super constructor A () is undefined. Must explicitly invoke another constructor]
OOPS294	Compilation Error	[Error: The constructor A () is undefined] you must pass some value on object creation.
OOPS295	A(int) Cons B-> DC	Here, We are passing value with super constructor.
OOPS296	Compilation Error	[Error: Implicit super constructor A () is undefined. Must explicitly invoke another constructor] Constructor call must be the first statement in a constructor

Prog No.	Output	Error and Details
OOPS297	A->DC B-> DC B(int)	1st super class then sub class constructors will execute.
OOPS298	A->DC B(int)	we are passing value with object creation so default constructor of subclass will not executed.
OOPS299	Compilation Error	[Error : Constructor call must be the first statement in a constructor] Refer point 22, 23 and 24.
OOPS300	Compilation Error	[Error : Same””]
OOPS301	A(int) Cons B(int) Cons IND 10	Passing 2 arguments with object creation.
OOPS302	Compilation Error	[Error : Cannot refer to 'this' nor 'super' while explicitly invoking a constructor] ‘this’ keyword must be used after invoking super class constructors only.

Write your note here……